

Recomendadores Generativos II: tokenización de ítems – RQ-VAE y Semantic IDs

IIC3633 – Sistemas Recomendadores

Denis Parra

Profesor Asociado

PUC Chile

Contenidos

1. ¿Cómo indexar un ítem? IDs en RecSys vs. tokens en LLMs
2. Tres familias de tokenización: Sparse ID, Text ID, Semantic ID
3. Fundamentos: VQ-VAE y RQ-VAE
4. TIGER: Semantic IDs para generative retrieval
5. Semantic IDs en ranking: el trabajo de Singh, ... Ed Chi, Yi (Google/YouTube)
6. Técnicas y entradas para construir Semantic IDs
7. Desafíos abiertos y laboratorio sugerido

Recordatorio de la clase 1

- Un LLM congelado no basta: se necesita **conocimiento colaborativo**
- Una vía fue agregar **tokens de ítem** al vocabulario del LLM (LC-Rec)
- Pregunta de hoy: **¿cómo representamos cada ítem para que un modelo generativo pueda producirlo?**
- Es la primera de las tres decisiones de diseño de GenRec: **tokenización** → arquitectura → optimización

¿Cómo se indexa un ítem en RecSys?

- Cada ítem tiene un **ID único** (ej.: B097B2YWFX), sin significado
- Ejemplo: **SASRec** (Kang & McAuley, ICDM 2018) aprende un embedding por ID y modela la secuencia de ítems del usuario
- Tabla de embeddings y índice ANN crecen linealmente con el catálogo
- Átomos sin semántica compartida: mala transferencia a ítems de cola larga o nuevos

¿Y en un LLM?

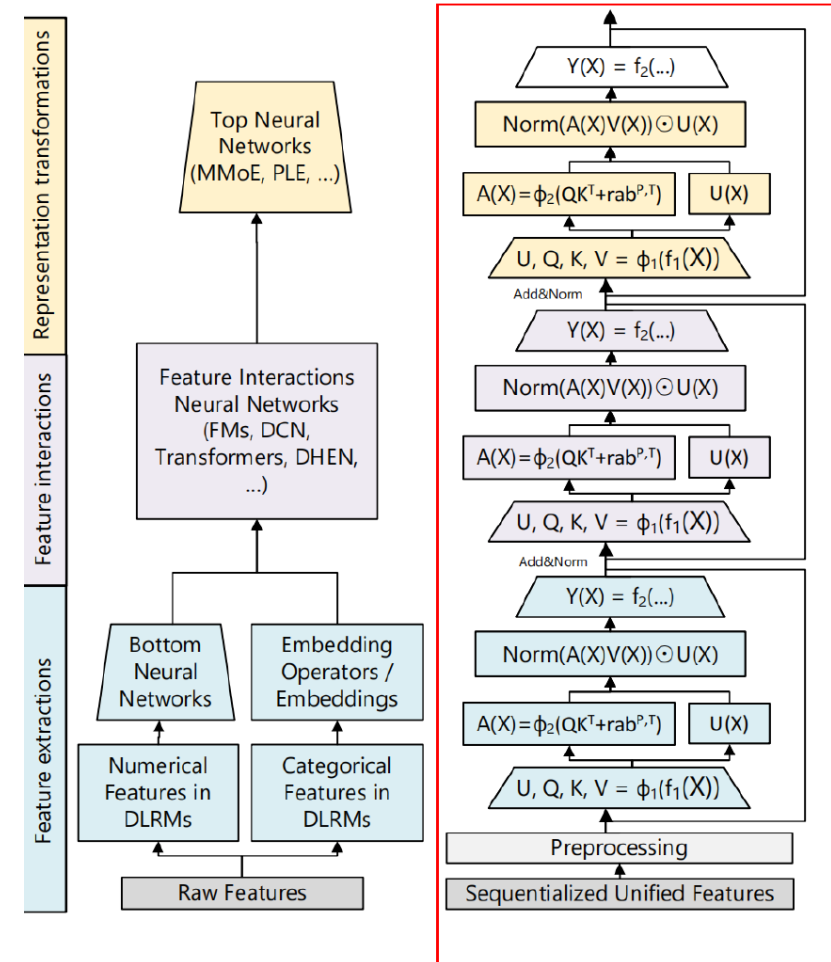
- Vocabulario de un LLM: ~128 mil a ~256 mil tokens
- Catálogo (Amazon Reviews 2023): ~48,2 millones de ítems
- Es difícil alinear ambos vocabularios: un ID por ítem no cabe en el vocabulario del LLM
- **Pregunta clave:** ¿se puede indexar un gran volumen de ítems con un **vocabulario compacto**?

Tres familias de tokenización

Familia	Representación	Ventajas	Limitaciones	Ejemplos
Sparse ID	ID aleatorio único; vocabulario masivo	Sin colisiones; interacción de features; listo para industria	Sin semántica; cold-start	HSTU, MTGR
Text ID	Descripción en lenguaje natural	Conocimiento del mundo; zero/few-shot; interpretable	Largo variable; difícil generar el ítem exacto	TALLRec, P5
Semantic ID	Secuencia corta y fija de códigos discretos	Compacto, semántico, coarse-to-fine	Colisiones; objetivo de reconstrucción $\neq$ recomendación	TIGER, LC-Rec, LETTER, OneRec

Sparse ID: HSTU

- Reformula ranking y retrieval como **transducción secuencial**: [ítem₁, acción₁, ..., ítem_n, acción_n]
- Atención agregada puntual (reemplaza softmax); $5,3\times$ a $15,2\times$ más rápido que Transformers con FlashAttention2
- Hasta +65,8% NDCG sobre SASRec en datasets públicos; +12,4% en A/B online
- **MTGR** (Meituan): conserva cross-features de IDs dispersos dentro de un esquema generativo tipo HSTU



Text ID: TALLRec y P5

- Representan el ítem con su texto (“Title: ...”, “Brand: ...”) usando el vocabulario del LLM
- Ventajas: integra conocimiento del mundo, resuelve cold-start y permite transferencia entre dominios
- Limitaciones: textos de largo variable; el modelo debe reproducir exactamente un título; sin estructura jerárquica ni señal colaborativa
- Motiva los **Semantic IDs**: ¿una representación compacta, jerárquica y aprendible?

Semantic IDs (SemID / SID)

- Pocos tokens que **juntos** indexan un ítem, ej.: (t₃, t₃₂₁, t₆₄₃, t₁₀₁₁)
- Los tokens salen de un **vocabulario compartido por todos los ítems**, por nivel
- Capacidad: 4 tokens con vocabulario 256 $\rightarrow 256^4 \approx 4,3 \times 10^9$ ítems con solo 1.024 tokens
- Ítems similares comparten prefijos: estructura **gruesa** $\rightarrow$ **fina**
- Permiten ver la recomendación como una generación secuencia a secuencia

La cuenta que motiva todo: ¿cuántos parámetros cuesta un ítem?

Tabla de embeddings (ALS, SASRec)

Una fila por ítem
 $1.000.000 \text{ ítems} \times 128 \text{ dim}$
 $= 128.000.000 \text{ de parámetros}$

Crece **lineal** con el catálogo

Semantic IDs (TIGER, LC-Rec)

Un vocabulario compartido
 $4 \text{ niveles} \times 256 \text{ códigos} \times 128 \text{ dim}$
 $= 131.072 \text{ parámetros}$

Constante: el catálogo puede crecer sin agrandar el modelo

≈ 1.000 veces menos parámetros para representar el mismo catálogo

- Pero el ahorro no es gratis: muchos ítems **comparten** tokens, así que el modelo ya no puede memorizar cada ítem por separado
- Ese es justamente el objetivo: forzar a que ítems parecidos compartan parámetros, y que uno nuevo herede lo aprendido por sus vecinos
- La pregunta de esta clase es cómo elegir esos códigos para que compartir ayude en vez de confundir

Elaboración propia. Las dimensiones son las típicas de la literatura (128 por embedding; 4×256 en TIGER).

De la cuantización vectorial a RQ-VAE

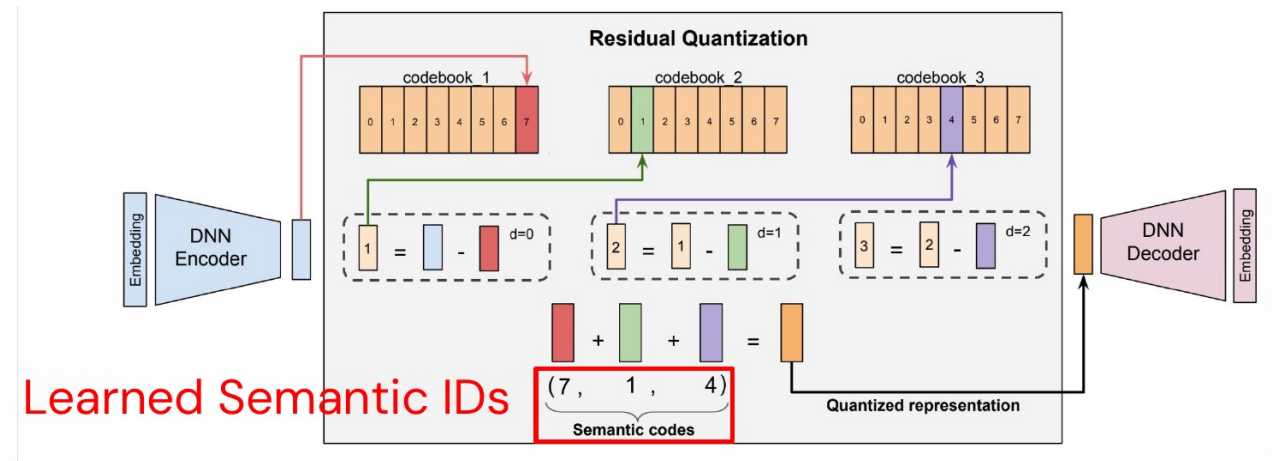
FUNDAMENTOS

Cuantización vectorial: VQ-VAE

- **VQ-VAE** (van den Oord et al., 2017): el encoder produce un vector continuo z ; se reemplaza por el vector más cercano de un **codebook** $\{e_k\}$
- Resultado: representación **discreta** (el índice k) que un decoder reconstruye
- Variantes para generación: VQ-GAN, Parti, SoundStream
- Limitación: un solo codebook exige un K enorme para describir bien los vectores

RQ-VAE: cuantización residual

- **RQ-VAE** (Lee et al., CVPR 2022; basado en el cuantizador residual de SoundStream, Zeghidour et al. 2021)
- Idea: cuantizar el **residuo** en varios niveles; cada nivel refina el anterior
- Nivel 0: se busca el código más cercano a z ; residuo $r_1 = z - e(c_0)$
- Nivel 1: se cuantiza r_1 con otro codebook; $r_2 = r_1 - e(c_1)$; y así sucesivamente
- El ítem queda descrito por la tupla de índices $(c_0, c_1, \dots, c_{m-1}) =$ su **Semantic ID**



RQ-VAE: formulación y entrenamiento

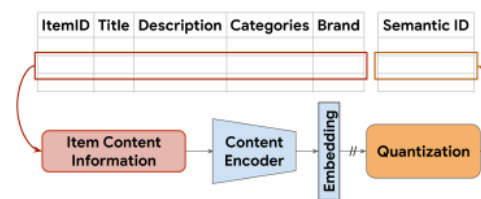
- Encoder E, cuantizador residual de L niveles (codebook C_l de tamaño K) y decoder D; $z = E(x)$, $r_0 = z$
- Código: $c_l = \operatorname{argmin}_k \|r_l - e_k\|$; siguiente residuo $r_{l+1} = r_l - e(c_l)$; $\hat{z} = \sum_l e(c_l)$; $\hat{x} = D(\hat{z})$
- Pérdida: $\mathbf{L} = \mathbf{L}_{\text{recon}} + \mathbf{L}_{\text{rqvae}}$, con $\mathbf{L}_{\text{recon}} = \|x - \hat{x}\|^2$ y $\mathbf{L}_{\text{rqvae}} = \sum_l \beta \|r_l - \operatorname{sg}[e(c_l)]\|^2 + \|\operatorname{sg}[r_l] - e(c_l)\|^2$ (sg = stop-gradient)
- Un codebook por nivel (la norma del residuo decrece); k-means en el primer batch para evitar *codebook collapse*

Generative retrieval con Semantic IDs

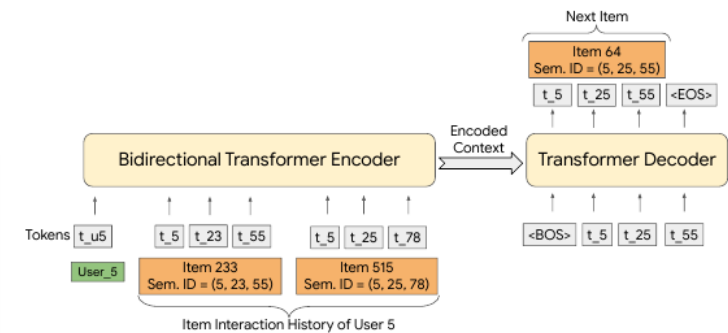
TIGER

TIGER: la idea

- Rajput, Mehta, Singh, ... Sathiamoorthy (Google), NeurIPS 2023
- Primer recomendador generativo basado en SIDs: predecir el **Semantic ID del siguiente ítem** en lugar de buscar vecinos en un índice ANN
- Entrada: SIDs de los ítems con que el usuario interactuó; salida: SID del próximo ítem
- Aporta: mejor recall/NDCG, cold-start y diversidad controlable



(a) Semantic ID generation for items using quantization of content embeddings.



(b) Transformer based encoder-decoder setup for building the sequence-to-sequence model used for generative retrieval.

TIGER paso a paso

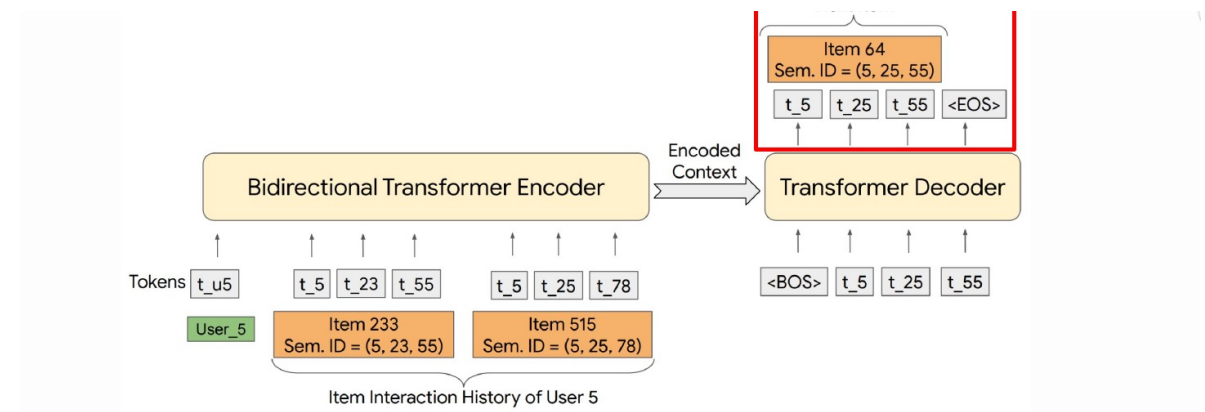
- **1. Contenido del ítem:** título, descripción, categorías, marca
- **2. Encoder de contenido:** Sentence-T5 congelado → embedding de 768 dimensiones
- **3. RQ-VAE:** encoder MLP (512-256-128), 3 niveles con codebooks de 256 vectores de dim. 32, $\beta = 0,25$ → código de 3 tokens
- **4. Colisiones:** un 4º token desambigua ítems con el mismo código
- **5. Modelo:** Transformer encoder-decoder (T5) sobre secuencias de SIDs; token de usuario con hashing (2.000 tokens)

Colisiones

- Dos ítems distintos pueden obtener el mismo código, ej.: (12, 24, 52)
- Solución en TIGER: agregar un **token extra** al final: (12, 24, 52, 0) y (12, 24, 52, 1)
- Se detecta con una tabla de búsqueda código → ítems, una sola vez, tras entrenar el RQ-VAE
- Alternativas modernas: restricciones en el clustering, Sinkhorn, token con el ID disperso (CAR)

Propiedades de los SIDs basados en RQ-VAE

- **Semánticos:** ítems con contenido parecido comparten prefijo
- **Ordenados:** dependencia secuencial coarse-to-fine, alineada con la decodificación autoregresiva
- **Compactos:** poco vocabulario para catálogos enormes
- **Cold-start:** ítems nuevos reciben un SID desde su contenido
- **Diversidad controlable:** muestreo con temperatura al decodificar
- Resultados reportados: hasta +29% NDCG@5 vs. SASRec y +17,3% Recall@5 vs. S³-Rec en datasets de Amazon



¿Cuánto aporta un Semantic ID? Evidencia en TIGER

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	<u>0.0249</u>	0.0605	0.0318	<u>0.0463</u>	<u>0.0306</u>	0.0675	0.0374
S ³ -Rec [44]	<u>0.0251</u>	<u>0.0161</u>	<u>0.0385</u>	<u>0.0204</u>	<u>0.0387</u>	0.0244	<u>0.0647</u>	<u>0.0327</u>	0.0443	0.0294	<u>0.0700</u>	<u>0.0376</u>
TIGER [Ours]	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

Amazon Beauty, Sports y Toys. TIGER supera a SASRec, S³-Rec, BERT4Rec y P5 en Recall y NDCG; en Beauty, NDCG@5 pasa de 0,0249 (SASRec) a 0,0321.

La parte que importa es la semántica, no la tokenización en sí

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
Random ID	0.007	0.005	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH SID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE SID	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432

Mismo modelo, tres maneras de construir el identificador: IDs aleatorios < SIDs por LSH < SIDs por RQ-VAE. Si los códigos no capturan contenido, el modelo generativo pierde su ventaja.

Tabla 2 de Rajput et al. NeurIPS 2023.

Cómo leer estas dos tablas juntas

- La primera dice que **el enfoque** funciona; la segunda dice **por qué** funciona
- Comparar siempre contra el baseline correcto: un SASRec bien ajustado sigue siendo fuerte y en varios estudios posteriores la brecha se achica
- El experimento del laboratorio (mini-TIGER) reproduce el orden de magnitud de la columna de Beauty, no los valores exactos: menos épocas, catálogo recortado y cuantizador más simple
- Lo que un estudiante debería poder responder: si cambio RQ-VAE por códigos aleatorios, ¿qué columna de la tabla 2 predice lo que va a pasar?

Del retrieval generativo al ranking en YouTube

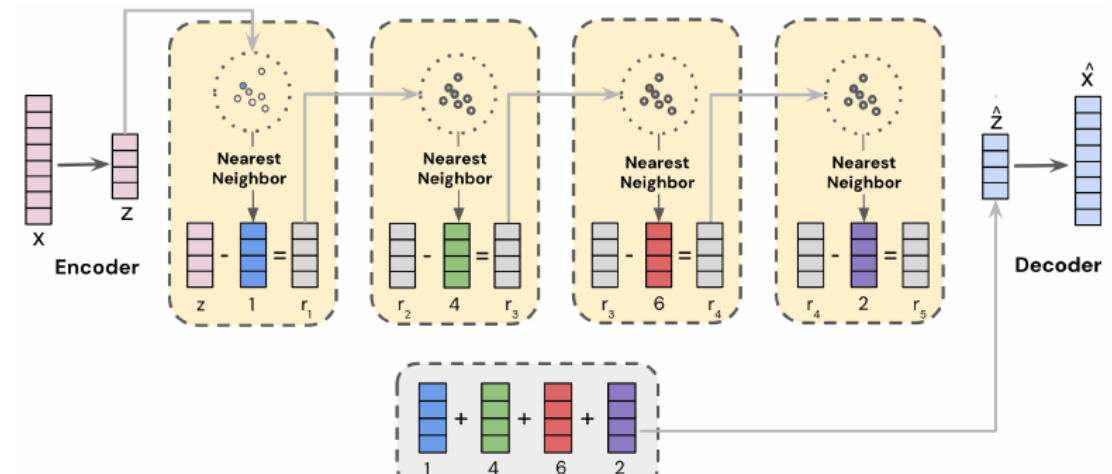
SEMANTIC IDS (ED CHI ET AL.)

Motivación: el problema de los IDs aleatorios

- Singh, Vu, Mehta, Keshavan, Sathiamoorthy, Zheng, Hong, Heldt, Wei, Tandon, **Ed Chi**, Yi (Google). RecSys 2024
- Los modelos de ranking industriales usan IDs de video **hasheados al azar** (decenas de millones de buckets para cientos de millones de videos)
- Colisiones aleatorias entre videos sin relación; mala generalización en la cola larga y en ítems nuevos
- Reemplazar el ID por embeddings de contenido **empeora** (Dense Input): se pierde memorización a nivel de ítem

Propuesta: dos etapas

- **Etapa 1:** comprimir embeddings de contenido en SIDs con RQ-VAE (embedding de video de 2.048 dim.; $L = 8$ niveles, $K = 2.048$); el RQ-VAE se **congela**
- **Etapa 2:** entrenar el modelo de ranking con embeddings aprendidos de los SIDs
- Los SIDs son jerárquicos: el nivel de prefijo controla la granularidad
- Un RQ-VAE entrenado con datos antiguos rinde de forma comparable a uno con datos recientes (estabilidad)



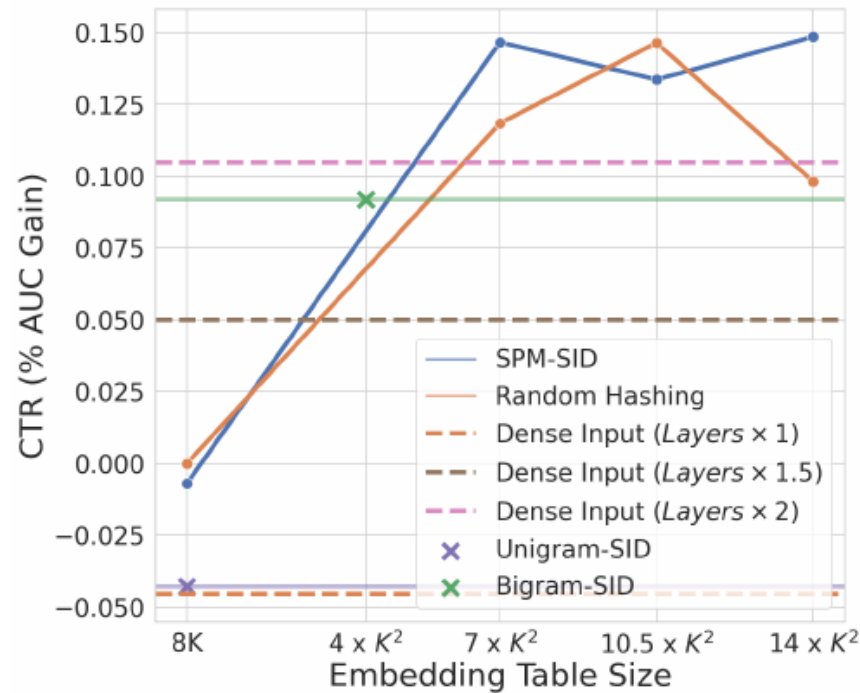
¿Cómo usar un SID en un ranker?

- Un SID largo no se puede mapear a una tabla de embeddings: hay que **hashear sub-piezas** de la secuencia
- **N-gram**: subpalabras de N códigos; unigrama = L filas por nivel, bigrama = K^2 filas por grupo; tabla crece exponencialmente (se limita $N \leq 2$)
- **SentencePiece (SPM)**: subpalabras de largo variable aprendidas de la distribución de impresiones; co-ocurrencias frecuentes se fusionan, las raras caen a unigrama
- SPM permite fijar el tamaño de tabla y **balancear memorización y generalización**

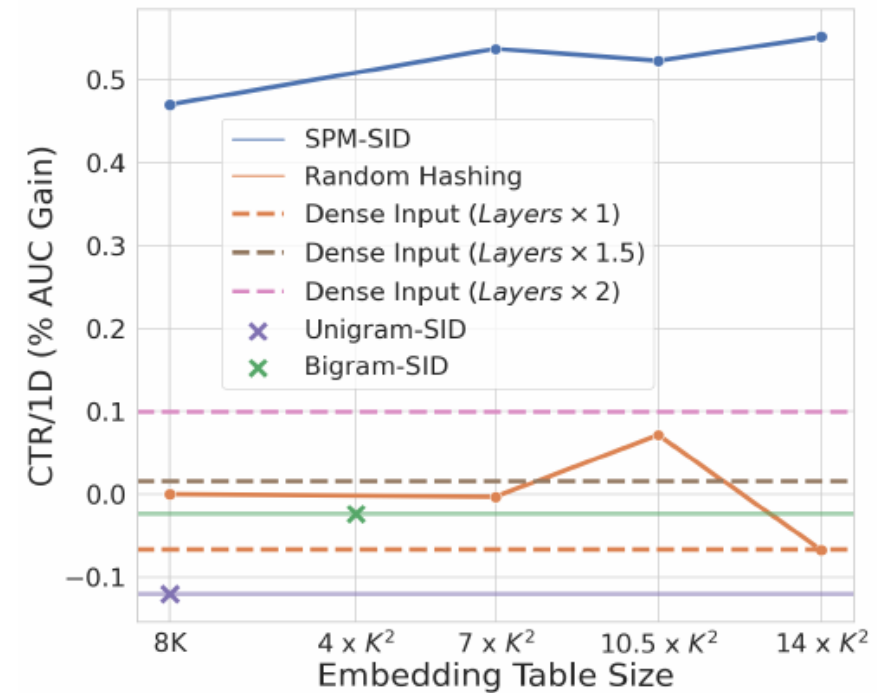
Resultados en YouTube

- Modelo de ranking multitarea para “siguiente video”; métricas: AUC de CTR global y CTR/1D (videos del día)
- Content embeddings directos < random hashing; más capas ayudan pero elevan costo de servicio
- Con historial de usuario, SID unigrama y bigrama superan al hashing; **SPM** > **n-gram** con tablas grandes y mejora el cold-start
- SIDs puestos en producción para billones de videos
- Lección: los SIDs sirven no solo para **generar** (TIGER) sino también para **rankear** con mejor generalización

Semantic IDs en producción: evidencia en YouTube



(a) Overall CTR AUC



(b) Cold-start CTR/1D AUC

Mejora porcentual de AUC de CTR frente a hashing aleatorio, según el tamaño de la tabla de embeddings. El panel derecho corresponde a videos del día (cold-start), donde la ganancia es mayor.

Dos maneras distintas de usar el mismo Semantic ID

- **Como identificador a generar** (TIGER, LC-Rec, OneRec): el modelo produce los tokens y el ítem sale de una tabla de búsqueda
- **Como atributo de entrada** (YouTube, Snapchat): el ranker de siempre reemplaza el ID aleatorio por el SID y gana generalización, sin cambiar la arquitectura
- La segunda vía es menos ambiciosa y mucho más fácil de desplegar: no toca el servicio de inferencia ni los índices
- Para un proyecto del curso, la segunda vía suele ser la que cabe en el tiempo disponible

Técnicas, entradas y variantes

CONSTRUIR SEMANTIC IDS

Técnicas para construir SIDs

Técnica	Idea	Ejemplos
Residual quantization (+ regularización)	Códigos por niveles; regularización por ítem o pérdida de recomendación	TIGER, ColaRec, ETEGRec
Product quantization	Particiones independientes (no ordenado)	VQ-Rec; OPQ en RPG
Clustering jerárquico	Heurístico o latente; ordenado, largo variable	Hua et al.; Si et al.
Generador basado en LM	Texto de entrada, SIDs de salida	LMIndexer (Jin et al.)
Dependientes del contexto	El mismo ítem puede tener distintos IDs según el contexto	ActionPiece

Fuente: Tutorial 'Large Generative Recommendation Models' (WWW 2025) (pp. 185-203)

RQ-KMeans y estabilidad del codebook

- **Codebook collapse:** pocos códigos concentran la mayoría de los ítems
- OneRec y OneLoc usan **ResKmeans** con k-means restringido: máximo de ítems por centroide, mejorando uso y estabilidad
- Sinkhorn con pérdida de entropía para asignación uniforme (SaviorRec)
- Riesgos: *hourglass effect* (niveles intermedios muy concentrados) y dependencia de prefijo

LETTER: SIDs aprendibles

- Wang et al., CIKM 2024. Mejora tokenizadores tipo TIGER/LC-Rec con tres regularizaciones:
- **Semántica:** integrar jerarquía semántica en la secuencia
- **Colaborativa:** alinear códigos cuantizados con embeddings de un CF entrenado (ej.: SASRec)
- **Diversidad:** distribuir los códigos uniformemente para evitar sesgo de asignación
- Además, pérdida de generación guiada por ranking (hard negatives); mejora TIGER y LC-Rec

¿Qué información entra al tokenizador?

- **Texto o multimodal:** título, descripción, imagen
- **Catóricas:** categoría, marca
- **Sin features:** solo estructura de interacciones
- **Metadata + comportamiento:** señal colaborativa en el embedding o en el cuantizador
- Ej.: **OneRec** (miniCPM-V-8B + QFormer + alineación colaborativa + RQ-KMeans de 3 niveles); **QARM** (alinea un MLLM con pares ítem-ítem de retrieval; códigos VQ y RQ)

Comparación de tokenizadores con SIDs

Método	Entrada	Cuantizador	Aporte
TIGER	Texto (Sentence-T5)	RQ-VAE	Primer SID generativo; token anti-colisión
LC-Rec	Texto + semántica colaborativa	RQ-VAE	Alinea SIDs con lenguaje del LLM
LETTER	Contenido + CF	RQ-VAE reg.	Regularizaciones colaborativa y de diversidad
RPG	Contenido	OPQ (hasta 64 tokens)	Predicción paralela de múltiples tokens
OneRec	Multimodal + colaborativo	RQ-KMeans (3 niveles)	Primer tokenizador SID industrial
QARM	Multimodal alineado	VQ + RQ	Códigos entrenables end-to-end

Fuente: Tutorial 'Generative Recommendation: Foundations and Frontiers' (KDD 2026) (pp. 31-49)

Desafíos abiertos

- **Inconsistencia:** tokenizador congelado, reconstrucción $\neq$ objetivo de recomendación $\rightarrow$ ETEGRec entrena tokenizador y recomendador de forma alternada
- **Interpretabilidad:** los tokens de ítem no están alineados con el lenguaje (OneRec-Think)
- **Modalidad:** features multimodales preentrenadas desalineadas del comportamiento (QARM)
- **Largo del código:** más tokens, más costo de decodificación (RPG)
- ¿Qué métrica de tokenización predice el rendimiento? ¿Zipf, entropía?

Laboratorio sugerido

- Calcular embeddings de contenido de ítems (ej.: MovieLens + sinopsis) con un modelo de texto
- Construir SIDs con **RQ-KMeans** (3 niveles) y medir colisiones y uso del codebook
- Entrenar un modelo secuencial pequeño que prediga el SID del siguiente ítem
- Comparar con SASRec: Recall/NDCG globales y en ítems de cola larga o nuevos
- Pregunta: ¿cómo cambia el rendimiento al variar niveles y tamaño del codebook?

Resumen y próxima clase

- Un SID es un código corto, jerárquico y compartido que indexa ítems: **compacto y semántico**
- **RQ-VAE** cuantiza residuos por niveles; TIGER lo usa para generar el próximo ítem
- El trabajo de Singh, ... Ed Chi, Yi muestra SIDs en **ranking** (SPM vs n-gram) con ganancias en cold-start
- **Próxima clase:** arquitecturas (encoder-decoder, decoder-only, difusión) y optimización (NTP, DPO, GRPO)

Referencias

- Rajput et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.
- Singh et al. Better Generalization with Semantic IDs. RecSys 2024.
- van den Oord et al. Neural Discrete Representation Learning. NeurIPS 2017.
- Lee et al. Autoregressive Image Generation using Residual Quantization. CVPR 2022.
- Zeghidour et al. SoundStream. TASLP 2022.
- Wang et al. LETTER: Learnable Item Tokenization. CIKM 2024. Zheng et al. LC-Rec. ICDE 2024.
- Zhai et al. HSTU. ICML 2024. Han et al. MTGR. CIKM. Hou et al. RPG. KDD 2025. Liu et al. ETEGRec. SIGIR 2025.
- Tutoriales WWW 2025 y KDD 2026 sobre Generative Recommendation.

Fin

- `denis.parra@uc.cl`